

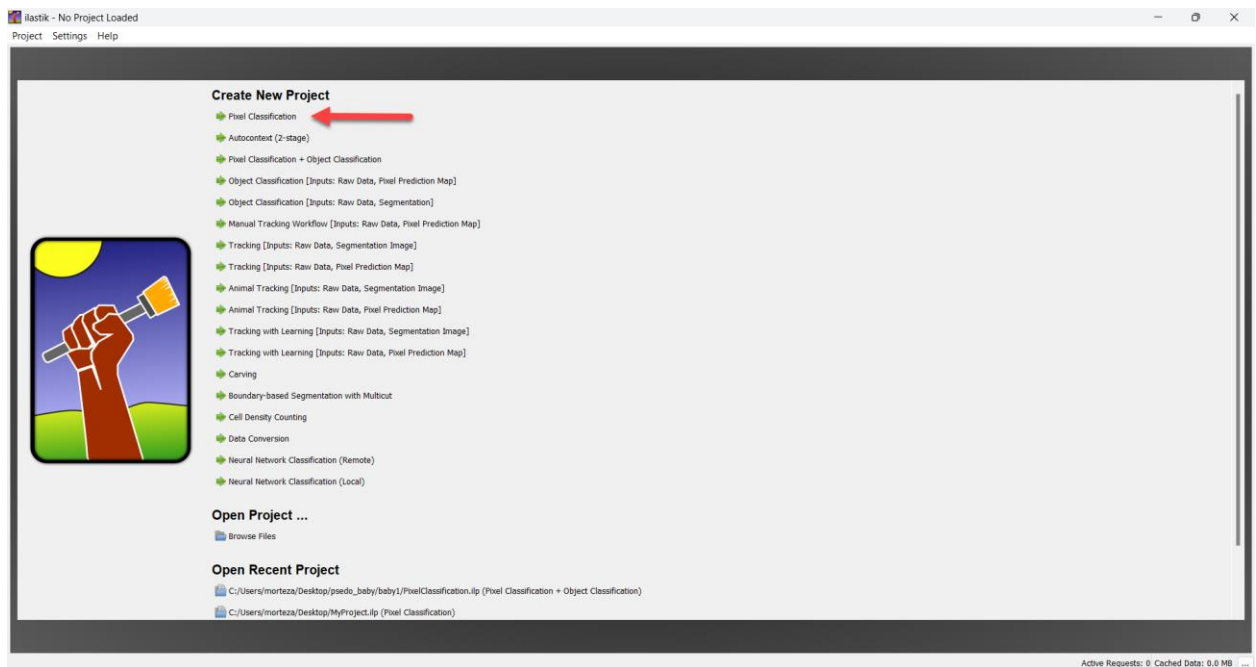
Pixel Classification workflow:

“The Pixel Classification workflow assigns labels to pixels based on pixel features and user annotations. The workflow offers a choice of generic pixel features, such as smoothed pixel intensity, edge filters and texture descriptors. Once the features are selected, a Random Forest classifier is trained from user annotations interactively. Random Forest is known for its excellent generalization properties, the overall workflow is applicable to a wide range of segmentation problems. Note that this workflow performs semantic, rather than instance, segmentation and returns a probability map of each class, not individual objects.” (Ilastik website)

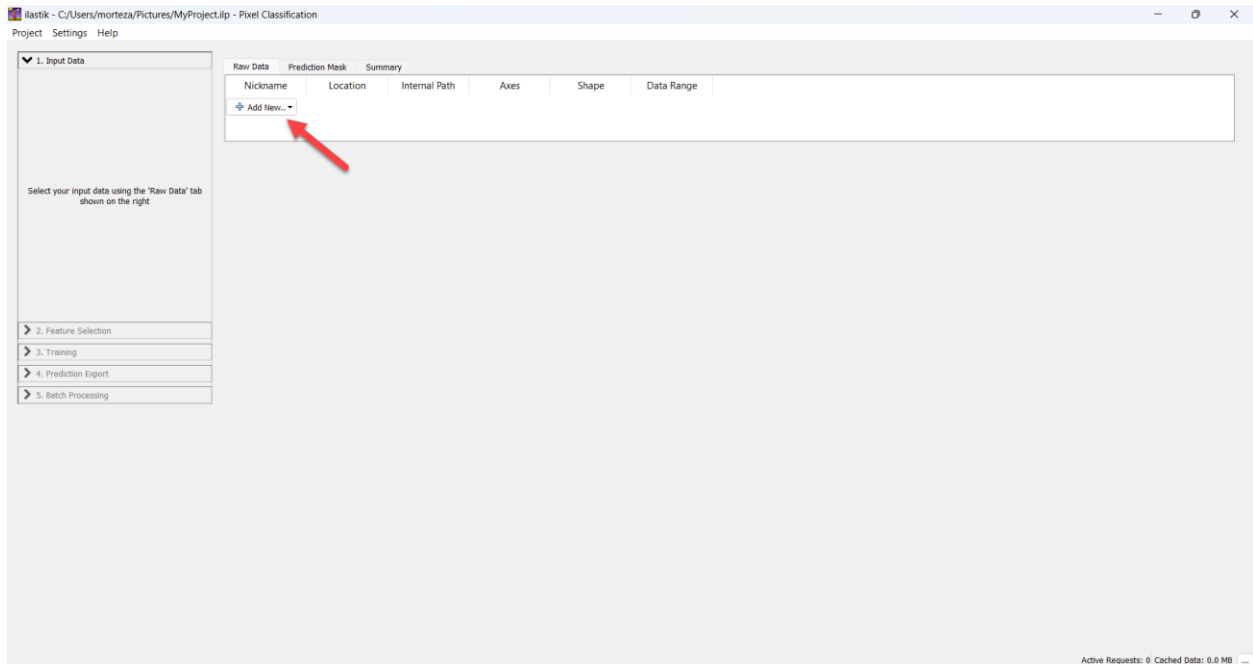
A typical cell segmentation use case is depicted below.

1. Create New Project

Click on **Pixel Classification**



2. Click on **Add New...** and import your raw images.



Note:

The sequence of raw images may not always be in numerical order. Consequently, before importing these images into Ilastik, you may need to adjust their names to maintain proper order. I've accomplished this by utilizing a straightforward Python script.

Let's assume you have a batch of 100 images, which means the maximum digit length of the image number would be three. Accordingly, the image naming should adhere to the following pattern:

For single-digit images, use: NAME00[0- 9]

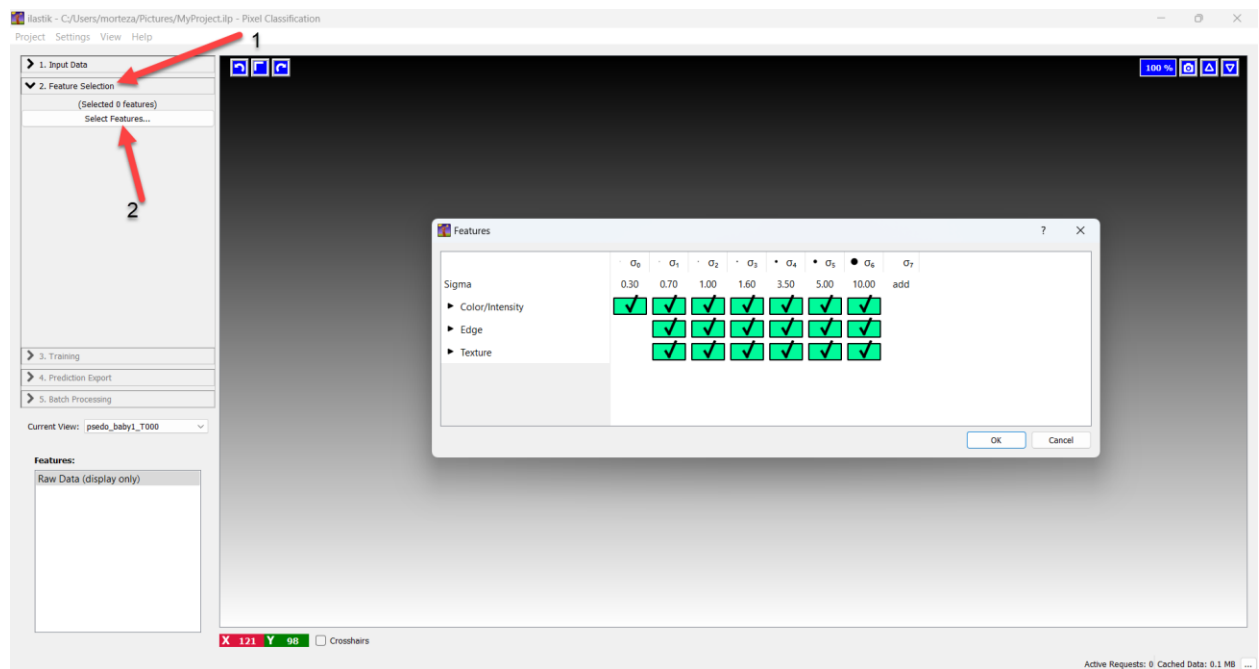
For two-digit images, use: NAME0[10- 99]

And for three-digit images use: NAME[100- 999]

3. Selecting good features

After the data is loaded, switch to the next applet **Feature Selection**. Here you will select the pixel features and their scales which in the next step will be used to discriminate between the different classes of pixels.

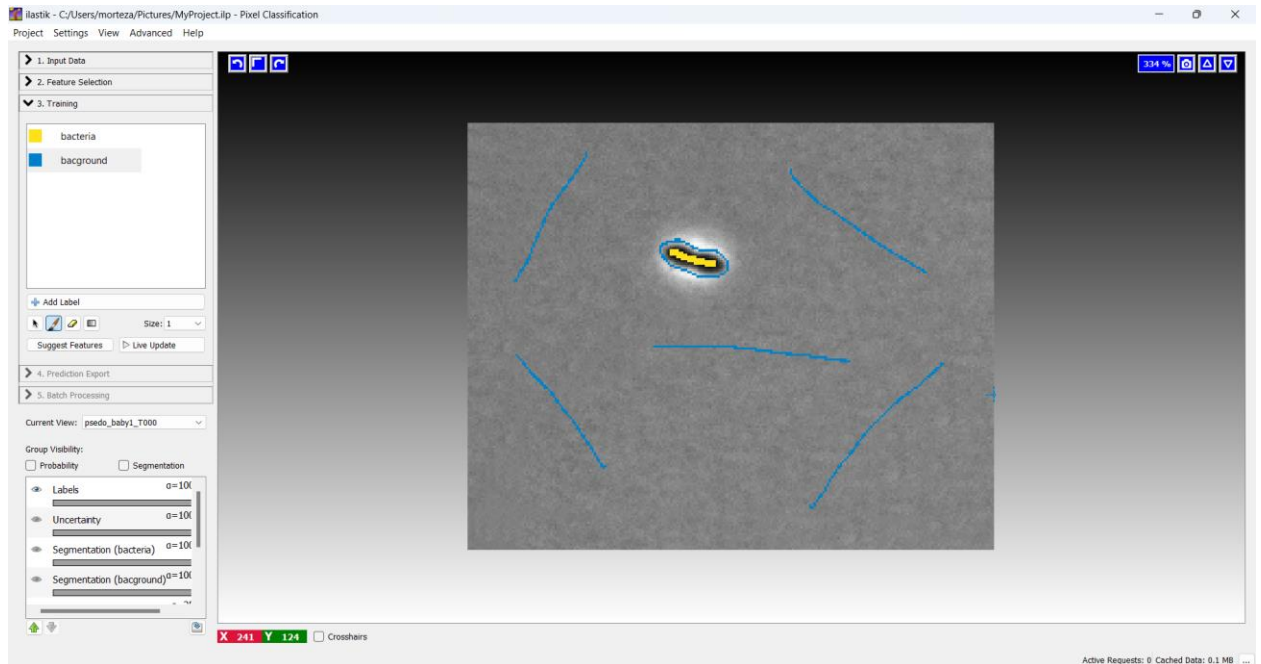
click on the **Select Features** button brings up a feature selection dialog. I selected all of the features.



4. Training the classifier

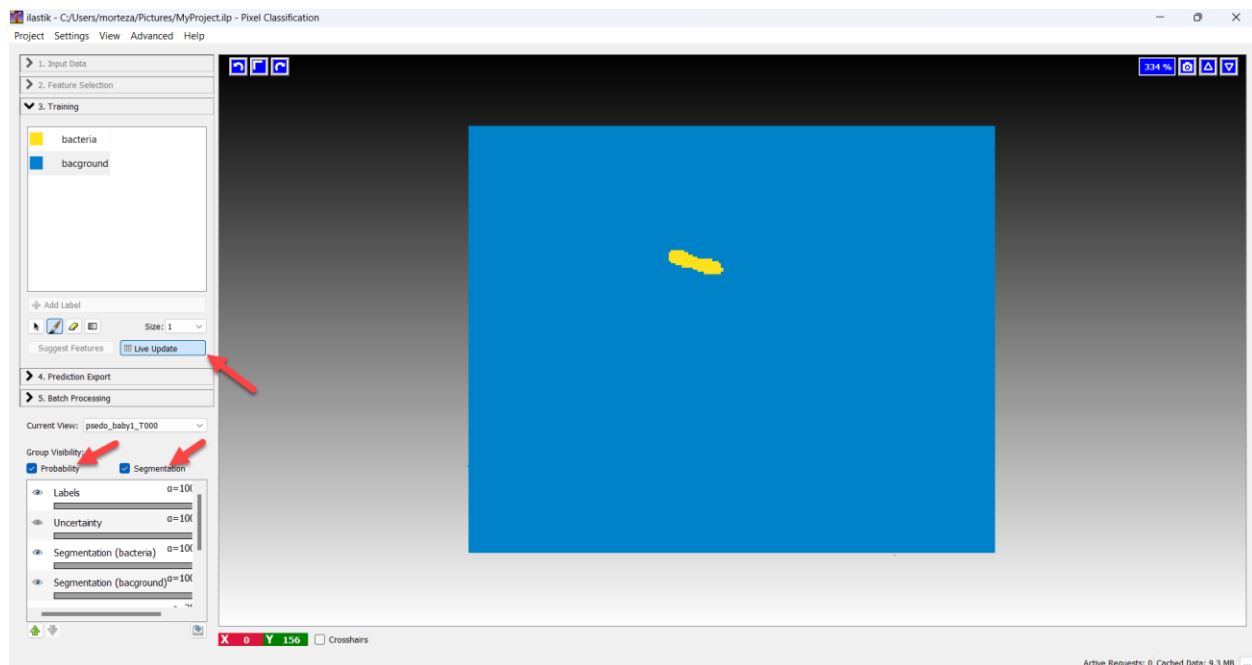
The next step in the pixel classification is the training of a classifier that can separate the object classes. This training is done in an iterative fashion, the user draws some annotations, evaluates the interactive prediction and then draws additional annotations to correct eventual mistakes. To begin with the training of the classifier, we switch to the **Training** applet and add some labels.

I create two labels: one for bacteria (marked with yellow) and one for the background (marked with blue). Then, I color some pixels representing bacteria with yellow and some pixels representing the background with blue.

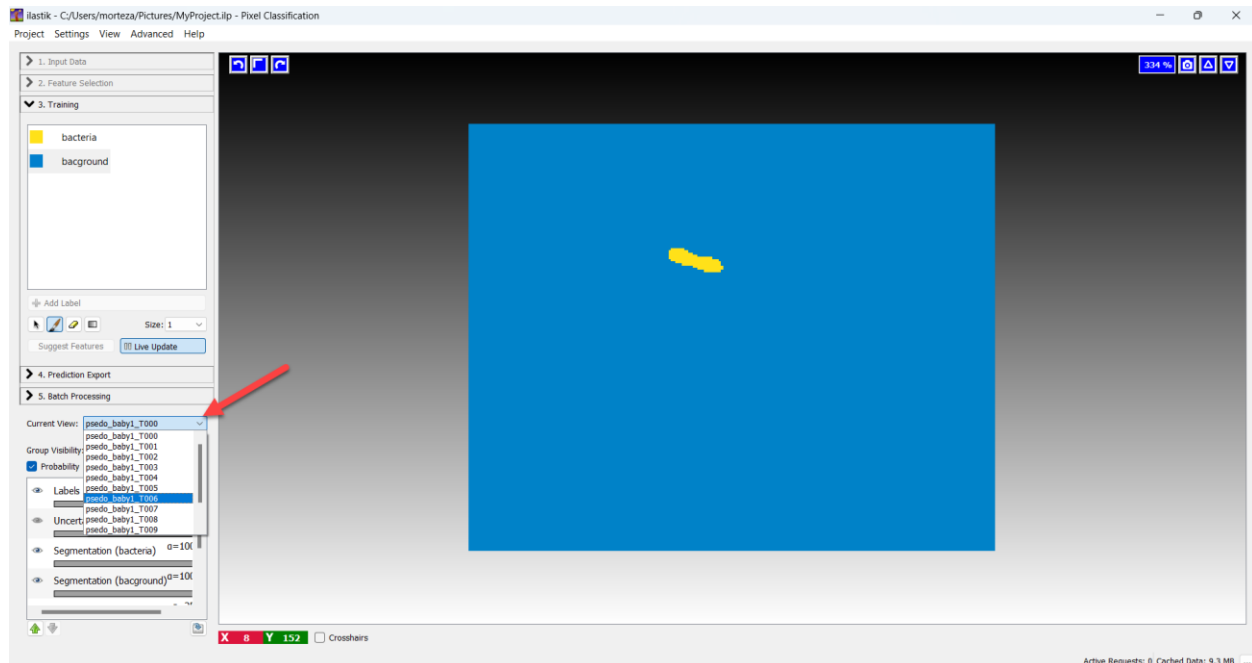


Afterward, I click 'Live Update' to see the trained model's output.

To see the model output, I select **Probability** & **Segmentation** from Group Visibility.



If the results aren't right, I fix the wrong-colored pixels with their true colors. Then, I go to a new time step by clicking on 'Current View' and picking a different time step. I keep fixing colors, checking results, and changing time steps until the model looks good.

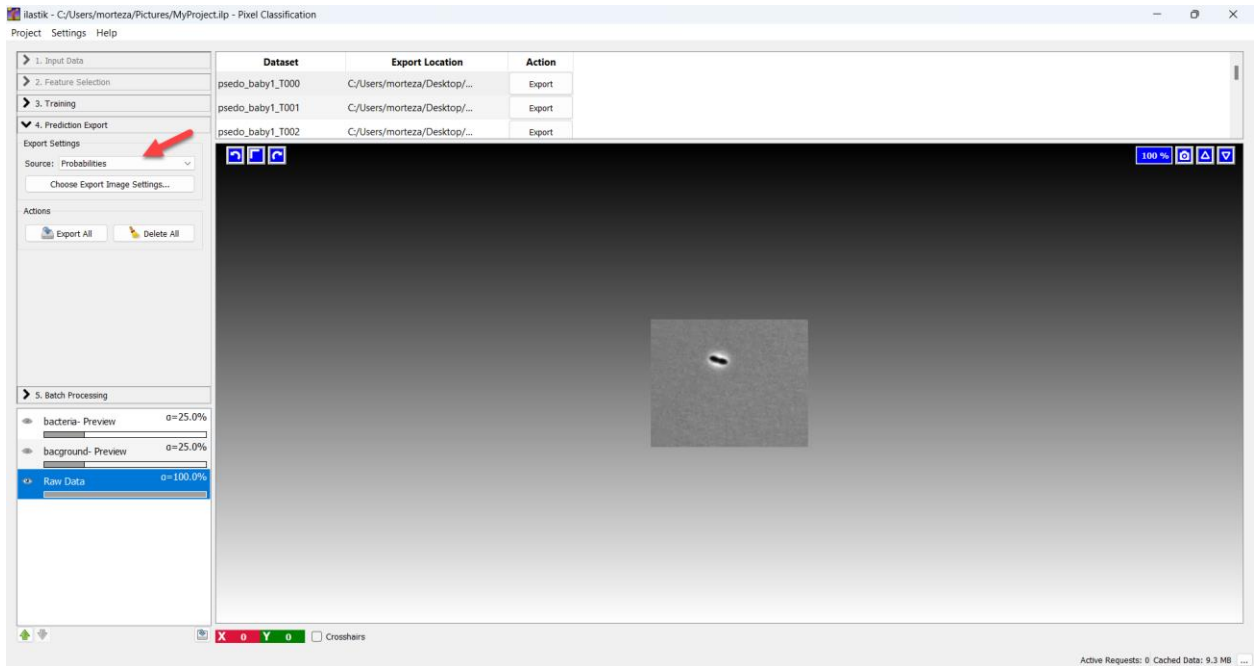


5. Exporting Output

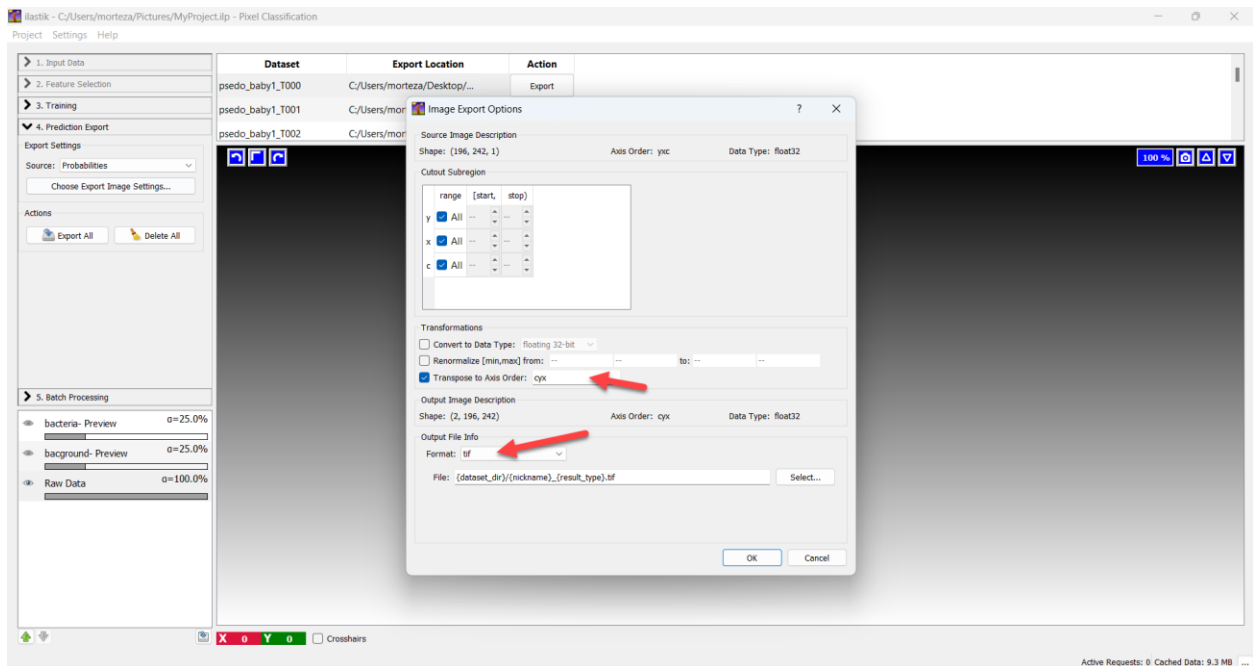
The export step is handled through the data export applet in Ilastik. In pixel classification, the applet is called “**Prediction Export**”.

The GUI can be used to change export settings as well as to bulk export all of the data.

I choose '**Probabilities**' as the source of the output.



Then I click on 'Choose Export Image Settings...' and define 'cyx' as the axis order and 'tif' as the desired output format.



Lastly, I click on the "Export All" option to export the images.

Contact information:

Atiyeh Ahmadi: a94ahmad@uwaterloo.ca

Dr. Brian Ingalls: bbingalls@uwaterloo.ca